

DISPENSA DELLA LEZIONE

Fondamenti di JavaScript

Script, variabili, tipi di dato, stringhe e operazioni numeriche.

CAPITOLO 1

Inserire uno script in una pagina HTML

NOTA

Il tag `<script>` può contenere JavaScript scritto direttamente nella pagina, collegare un file esterno con `src` oppure ospitare dati non eseguibili, come JSON.

Il tag `<noscript>` mostra un contenuto alternativo quando JavaScript non è disponibile o non è attivo.

Uno script può essere inserito in tre modi principali: direttamente nella pagina, in un file separato oppure su un singolo tag tramite un evento. La scelta dipende da quanto codice devi scrivere e da quanto vuoi tenere ordinato il progetto.

Il codice inline è utile per esempi semplici, il file separato è più comodo quando il progetto cresce, mentre il codice legato a un evento serve per azioni rapide su un elemento preciso.

Esempio pratico

```
<!-- Script in pagina -->
<script>
    console.log('Messaggio scritto direttamente nella pagina');
</script>

<!-- Script in file separato -->
<script src="assets/main.js"></script>

<!-- Script collegato a un singolo elemento -->
<button onclick="console.log('Click sul pulsante')">Premi qui</button>
```

CAPITOLO 2

Tipi di script e attributi utili

NOTA

`type="text/javascript"` è la forma classica, `type="module"` abilita i moduli e

`type="application/json"` serve per inserire dati JSON non eseguibili.

`defer` rinvia l'esecuzione fino a quando l'HTML è stato analizzato, mentre `async` esegue il file appena è disponibile.

Alcuni attributi cambiano il comportamento del tag `<script>`. `type="module"` permette di usare il sistema dei moduli, mentre `type="application/json"` è utile quando vuoi conservare dati dentro la pagina senza eseguirli.

`defer` è adatto ai file esterni che devono aspettare la fine del parsing dell'HTML. `async` è utile quando il file può partire appena viene scaricato, senza dipendere dall'ordine della pagina.

Esempio pratico

```
<script src="assets/main.js" defer></script>

<script type="module">
  console.log('Questo script usa il tipo module');
</script>

<script type="application/json" id="config">
  {
    "tema": "chiaro",
    "lingua": "it"
  }
</script>
```

CAPITOLO 3

Variabili in JavaScript

NOTA

In JavaScript moderno si usano soprattutto `let` e `const`. `var` è una sintassi storica e conviene evitarla nei nuovi progetti.

Una variabile è un contenitore che salva un valore. Con `let` dichiari un valore che può cambiare, mentre con `const` dichiari un valore che non deve essere riassegnato. Questo aiuta a rendere il codice più chiaro e più sicuro da leggere.

`var` esiste ancora, ma oggi si preferisce non usarlo perché il suo comportamento è meno chiaro e meno utile per scrivere codice moderno.

Esempio pratico

```
let nome = 'Luca';  
const anno = 2026;  
let presente = true;  
  
nome = 'Marco';
```

CAPITOLO 4

Tipi di dato fondamentali

NOTA

I tre tipi base trattati sono `string`, `number` e `boolean`.

Una **string** è un testo, un **number** è un valore numerico e un **boolean** può valere solo `true` o `false`. Questi tipi servono per rappresentare informazioni diverse in modo corretto.

Quando scegli il tipo giusto, il codice diventa più leggibile e più facile da usare nelle operazioni successive. È importante distinguere tra testo e numero, perché non si comportano nello stesso modo.

Esempio pratico

```
const corso = 'JavaScript Base';  
const lezioni = 8;  
const online = true;
```

CAPITOLO 5

Concatenazione e template literal

NOTA

La concatenazione unisce stringhe con `+`, mentre i template literal usano i backtick e l'interpolazione con `${...}`.

Quando vuoi costruire una frase con più pezzi di testo, puoi unirli con la concatenazione. Con i template literal il risultato è più leggibile, soprattutto quando devi inserire variabili dentro una frase.

I template literal sono molto utili perché permettono di scrivere una stringa lunga senza interromperla in tanti pezzi separati.

Esempio pratico

```
const nome = 'Sara';

const messaggio1 = 'Ciao ' + nome + '! Benvenuta al corso.';
const messaggio2 = `Ciao ${nome}! Benvenuta al corso.`;
```

CAPITOLO 6

Operazioni algebriche sui numeri

NOTA

Con i numeri puoi usare somma, sottrazione, moltiplicazione, divisione e resto della divisione.

I valori numerici permettono di fare calcoli direttamente nel codice. Gli operatori principali sono `+`, `-`, `*`, `/` e `%`.

Questi operatori servono per sommare, togliere, moltiplicare, dividere e trovare il resto. Sono la base per qualsiasi calcolo semplice in JavaScript.

Esempio pratico

```
const a = 12;  
const b = 5;  
  
const somma = a + b;  
const differenza = a - b;  
const prodotto = a * b;  
const divisione = a / b;  
const resto = a % b;
```